

Flutter Navigation

A two-part course covering navigation in Flutter: first the built-in Navigator system (routes, push/pop, bottom navigation, tabs), then the modern declarative GoRouter package (route configuration, parameters, nested navigation with ShellRoute).

Instructor: Islam Amour Mohamed

FLUTTER
ROUTE
ROUTER

2
Parts

8
Modules

Course Roadmap

PART 01: BUILT-IN NAVIGATION

01 Introduction to Navigation

02 Navigator Basics

03 Bottom Navigation Bar

04 Tab Navigation

PART 02: NAVIGATION WITH GOROUTER

01 Introduction to GoRouter

02 Basic Navigation with GoRouter

03 Passing Data Between Routes

04 Nested Navigation with ShellRoute

Prerequisites: Flutter Static UI and Flutter Dynamic UI courses (widgets, layout, StatefulWidget, setState, forms)

PART 01

01

Built-in Navigation

Built-in Navigator Modules:

- Module 1: Introduction to Navigation
- Module 2: Navigator Basics
- Module 3: Bottom Navigation Bar
- Module 4: Tab Navigation

Introduction to Navigation Stack

1. What is navigation?

Navigation is how an app moves the user between different screens (also called pages or routes). Instead of one screen doing everything, an app is broken into focused screens, and navigation is the mechanism for transitioning between them.

Example: Tapping a product in a list to open its details screen, or tapping 'Sign In' to move from a login screen to a home screen, are both navigation events.

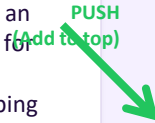
2. Navigation stack

Flutter manages screens using a stack data structure: the current screen is on top, and navigating to a new screen pushes it on top of the stack, while going back pops it off, revealing the previous screen underneath. Example: Stack after opening Home then Profile: [Home, Profile] (Profile on top). Going back pops Profile, leaving [Home].

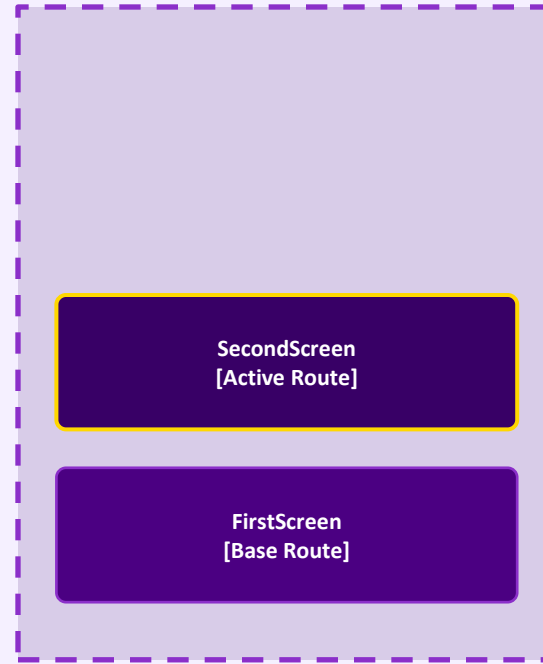
3. Routes and screens

A route represents a single entry on the navigation stack, typically corresponding to one screen/page widget. Each route can carry its own settings and data, and is what Navigator pushes and pops.

PUSH
(Add to top)



NAVIGATION STACK



POP
(Remove top)



Navigator & MaterialPageRoute

4. Navigator

Navigator is the widget that manages the stack of routes for an app (or a nested part of it). It exposes methods like `push()` and `pop()` to move between routes, and is usually already provided by `MaterialApp`.

5. MaterialPageRoute

`MaterialPageRoute` is a ready-made route type that provides the platform-appropriate transition animation (e.g. slide-in on Android) when navigating to a new screen built with a given widget.

MaterialPageRoute Push

```
Navigator.push(  
  context,  
  MaterialPageRoute(builder: (context) => SecondScreen()),  
);
```

Two-Screen Navigation

Create two screens and navigate between them using a button.

Requirements

- Create a FirstScreen with a button labeled 'Go to Second Screen'.
- Create a SecondScreen with a button labeled 'Go Back'.
- On FirstScreen's button press, use `Navigator.push()` with a `MaterialPageRoute` to open SecondScreen.
- On SecondScreen's button press, use `Navigator.pop()` to return to FirstScreen.

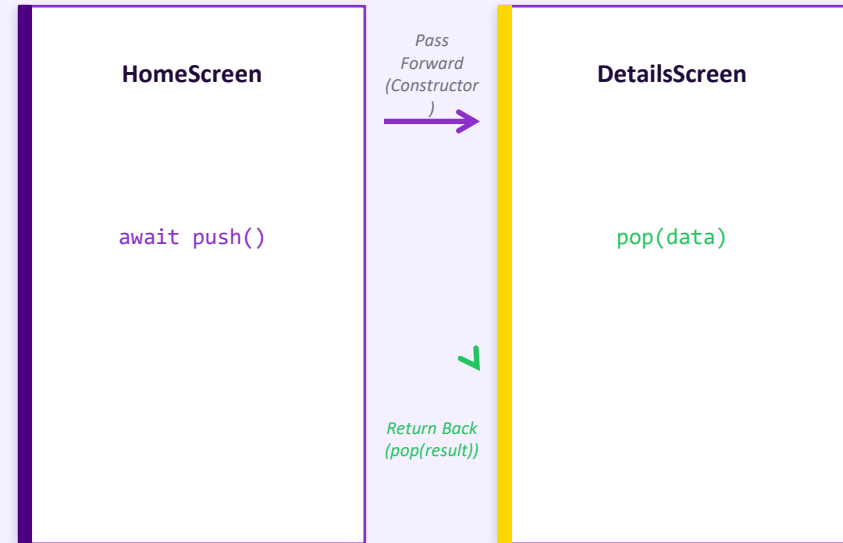
PRACTICE HINT

Wrap each screen in its own Scaffold so each has its own AppBar/body.

Navigator Basics & Data Passing

Navigator push() / pop() & Data flow

- `Navigator.push()` pushes a route onto the stack, transitioning forward.
- `Navigator.pop()` removes the top route, navigating back.
- Passing Data Forward: Constructor parameter.
- Returning Data Back: `pop(result)` returning result asynchronously.



Navigator replacements & cleared history

1. Navigator.pushReplacement()

`Navigator.pushReplacement()` replaces the current route with a new one instead of stacking on top of it, so the replaced screen is not reachable by pressing back — useful after login, where you don't want users returning to the login screen.

2. Navigator.pushAndRemoveUntil()

`Navigator.pushAndRemoveUntil()` pushes a new route and removes all previous routes that match a given predicate, commonly used to clear the entire navigation history (e.g. after logout, so back doesn't return into a previous session).

Replacement Routing

```
Navigator.pushReplacement(  
  context,  
  MaterialPageRoute(builder: (_) => HomePage()),  
);  
  
// Clear entire stack:  
Navigator.pushAndRemoveUntil(  
  context,  
  MaterialPageRoute(builder: (_) => LoginPage()),  
  (route) => false, // removes every previous route  
);
```


Simple Login Flow

Build a login flow using push, pushReplacement, and returning data with pop.

Requirements

- Build a LoginPage with a 'Log In' button that navigates to HomePage using `Navigator.pushReplacement()` (so Login isn't reachable via back button).
- On HomePage, add a button that navigates to ProfilePage using `Navigator.push()`.
- On ProfilePage, let the user pick or type a username, then return it to HomePage via `Navigator.pop(username)`.
- On HomePage, await the `push()` call and display the returned username once ProfilePage is popped.

PRACTICE HINT

Use `final result = await Navigator.push(...)` on HomePage to capture the value returned from ProfilePage.

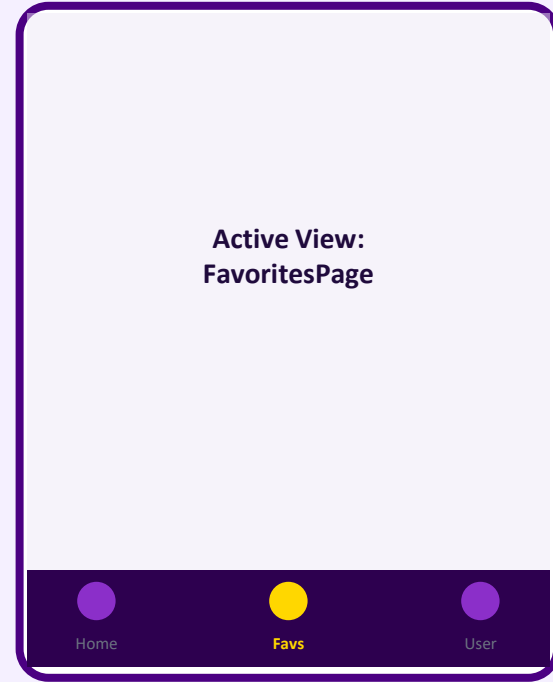
Bottom Navigation Bar & Tabs switching

1. BottomNavigationBar

BottomNavigationBar is a Material widget showing a row of tappable icons/labels fixed to the bottom of the screen, letting users switch between top-level sections of an app.

2. Selected Index & Page Switch

The active tab index is held in a StatefulWidget state variable. Tapping updates that index with setState(), redrawing the Scaffold body with corresponding widget.



IndexedStack & State Preservation

3. Preserving page state with IndexedStack

Directly swapping body: `pages[selectedIndex]` rebuilds a fresh page each time, losing scroll position or other in-page state.

`IndexedStack` keeps all pages built simultaneously (stacked), only showing the one at the given index, so switching tabs preserves each page's state.

4. When to use Bottom Navigation

Bottom Navigation suits apps with a small number (typically 3-5) of top-level, equally important destinations that users jump between frequently, rather than a linear flow (which suits push/pop navigation instead).

IndexedStack Navigation

```
IndexedStack(  
  index: selectedIndex,  
  children: [HomePage(), FavoritesPage(), ProfilePage()],  
)
```

Shopping App Tabs

Build a shopping app with three tabs, preserving each tab's state when switching.

Requirements

- Create a StatefulWidget shell with a BottomNavigationBar containing Home, Favorites, and Profile items.
- Track the selected tab index in state, updating it via onTap with setState().
- Display the current page using IndexedStack instead of directly swapping widgets, so state (e.g. scroll position) isn't lost.
- Confirm that scrolling down on Home, switching to Favorites, then switching back to Home preserves the scroll position.

PRACTICE HINT

IndexedStack(index: selectedIndex, children: [HomePage(), FavoritesPage(), ProfilePage()]) inside the Scaffold body.

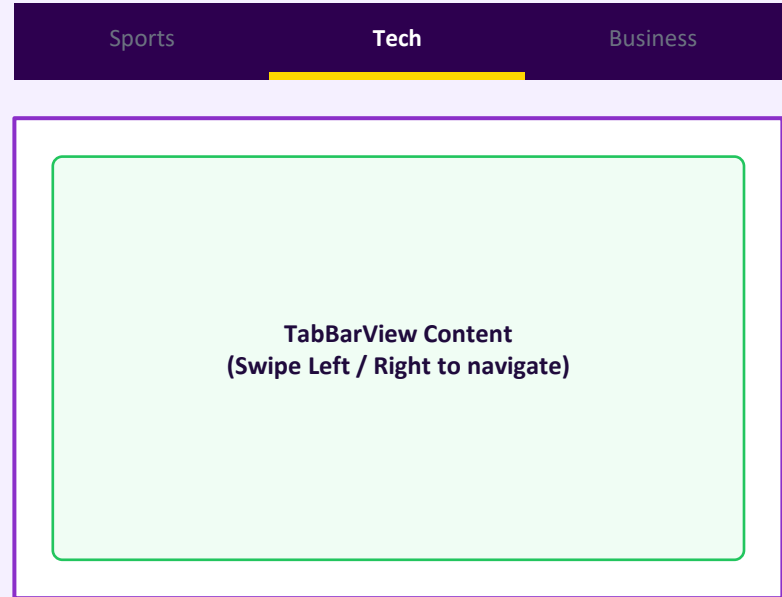
TabBar Viewport & Horizontal Swiping

1. DefaultTabController

DefaultTabController provides a shared TabController to its descendant TabBar and TabBarView widgets without you needing to manually create and manage one, based on a given length (number of tabs).

2. TabBar & TabBarView

- TabBar shows headers inside AppBar.bottom.
- TabBarView displays content pages.
- They swipe and sync automatically via DefaultTabController.



TabBar Customization & Tab vs Bottom Bar

3. Customizing tabs

Tabs can include icons alongside or instead of text, and their appearance (colors, indicator style) can be customized via `TabBar` properties like `labelColor`, `indicatorColor`, and `unselectedLabelColor`.

4. Tab navigation vs Bottom Navigation

`TabBar`/`TabBarView` is typically used for sub-sections within a single top-level screen (e.g. categories within a News screen) and supports horizontal swipe gestures, while `BottomNavigationBar` is used for top-level app sections and does not swipe between them by default.

DefaultTabController Setup

```
DefaultTabController(  
  length: 3,  
  child: Scaffold(  
    appBar: AppBar(bottom: TabBar(tabs: [...])),  
    body: TabBarView(children: [...]),  
  ),  
)
```

News Screen Tabs

Create a news screen with three tabs: Sports, Technology, and Business.

Requirements

- Wrap the news screen in a `DefaultTabController` with length: 3.
- Add a `TabBar` (inside the `AppBar`'s bottom) with tabs labeled Sports, Technology, and Business.
- Add a `TabBarView` with three corresponding pages, each showing placeholder content for that category.
- Confirm you can switch categories both by tapping a tab and by swiping horizontally.

PRACTICE HINT

DefaultTabController's length must match both the number of Tab widgets in TabBar and the number of children in TabBarView.

PART 02

02

Navigation with GoRouter

Declarative GoRouter Modules:

- Module 1: Introduction to GoRouter
- Module 2: Basic Navigation with GoRouter
- Module 3: Passing Data Between Routes
- Module 4: Nested Navigation with ShellRoute

Why GoRouter & Declarative Navigation

1. Why use GoRouter?

As apps grow, imperative `Navigator.push()/pop()` calls scattered throughout the code become hard to manage, especially for deep links, web URLs, and nested navigation. GoRouter offers a centralized, declarative way to define all routes in one place.

2. Declarative navigation

In declarative navigation, you describe the set of possible routes and the current location/state, and the router figures out what stack of screens to display — rather than imperatively pushing and popping routes step by step.

Example: Navigating declaratively means writing `context.go('/home')` and letting GoRouter determine the resulting screen, rather than manually constructing a `MaterialPageRoute` and calling `push()`.

Add GoRouter dependency in `pubspec.yaml`:

pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  go_router: ^14.0.0 # Or latest
```

GoRouter Setup & MaterialApp.router

3. Creating the router configuration

The router configuration is a GoRouter instance listing every GoRoute in the app: its path and the widget (builder) to display for that path.

4. MaterialApp.router

MaterialApp.router is a variant of MaterialApp designed to work with a router configuration (like GoRouter) instead of a simple home widget, wiring up route parsing and navigation for you.

GoRouter Setup

```
// Router config setup
final router = GoRouter(
  routes: [
    GoRoute(path: '/', builder: (context, state) => HomeScreen()),
    GoRoute(path: '/about', builder: (context, state) =>
      AboutScreen()),
  ],
);

// MaterialApp.router setup
MaterialApp.router(
  routerConfig: router,
);
```

Home & About with GoRouter

Configure GoRouter and navigate between Home and About screens.

Requirements

- Add the `go_router` package to the project.
- Create a `GoRouter` instance with two `GoRoutes`: `'/'` for `HomeScreen` and `'/about'` for `AboutScreen`.
- Replace `MaterialApp` with `MaterialApp.router`, passing the `GoRouter` instance as `routerConfig`.
- Add a button on `HomeScreen` that navigates to `'/about'`, and a button on `AboutScreen` that navigates back to `'/'`.

PRACTICE HINT

You'll use `context.go()` for these navigations, covered in detail in the next module.

GoRoute, Initial Location & Named Routes

1. GoRoute

GoRoute defines a single route in GoRouter's configuration: a path pattern and a builder function that returns the widget to show for that path. Routes can also be nested inside one another.

2. Initial Location & Named Routes

- Named Routes allow routing via name: `context.goNamed('settings')` which is less error prone.
- `initialLocation` specifies which path to open when the app boots (defaults to `/`).

Named Routes Config

```
// Route Config with names & initial location
GoRouter(
  initialLocation: '/splash',
  routes: [
    GoRoute(
      path: '/splash',
      name: 'splash',
      builder: (context, state) => SplashScreen(),
    ),
    GoRoute(
      path: '/home',
      name: 'home',
      builder: (context, state) => HomeScreen(),
    ),
  ],
);
```

context.go() vs context.push() & pop()

Context-based Navigation Methods

- `context.go()` updates the location to matches exactly, replacing stack if parent changes. Used for top-level pages (Home, Splash).
- `context.push()` stacks the page on top, keeping back stack. Used for sub-pages (details, settings).
- `context.pop()` removes the top route, returning back.

context.push()

MovieDetailsScreen
(Pushed)

HomeScreen

LoginScreen

Back stack preserved

context.go()

HomeScreen
(Target Location)

Stack replaced!

Splash, Login, Home Flow

Build a three-screen app: Splash, Login, and Home.

Requirements

- Configure three GoRoutes: '/splash', '/login', and '/home', with initialLocation set to '/splash'.
- On SplashScreen, after a short delay, use `context.go('/login')` to move to the login screen (replacing splash so back doesn't return to it).
- On LoginScreen, use `context.go('/home')` on successful 'login' to move to Home, replacing Login on the stack.
- From HomeScreen, add a button that uses `context.push('/login')` just to demonstrate push retains a back stack, then pop it with `context.pop()`.

PRACTICE HINT

Use `context.go()` for the splash->login->home flow (no back-stack desired) and `context.push()/pop()` only for the demonstration button.

Route Parameters: Path vs Query Parameters

1. Path parameters

Path parameters are dynamic segments in a route's path (prefixed with ':'), letting one route definition handle many specific screens, such as different product IDs.

2. Query parameters

Query parameters are optional key-value pairs appended to a URL after '?', useful for things like filters or sorting options that aren't part of the core path identity.

URL Structure Analysis

/movies/tt01111161?promo=halfprice

Path Param (:id)
Value: "tt01111161"
Used to identify a specific item.

Query Param (?promo)
Value: "halfprice"
Optional fields like sorting, filters, search tags.

Passing Complex Objects & Accessing Route State

3. Passing objects with extra

The extra parameter lets you pass an entire Dart object (not just a string) directly to the destination route, useful for passing a full model object instead of re-fetching it by ID.

4. Accessing route data

Inside a GoRoute's builder, the GoRouterState (commonly named state) exposes pathParameters, queryParameters (via state.uri), and extra, giving access to whatever data was passed during navigation.

Passing Object via extra

```
// Route registration with path parameters & extra
GoRoute(
  path: '/product/:id',
  builder: (context, state) {
    final id = state.pathParameters['id']!;
    final prod = state.extra as Product;
    return ProductDetailsScreen(id: id, product: prod);
  },
)

// Trigger navigation:
context.push('/product/123', extra: myProduct);
```


Product List & Details

Create a product list, open a details screen using a route parameter, and pass the full object using extra.

Requirements

- Create a ProductListScreen showing a list of sample products.
- Add a GoRoute at '/product/:id' for ProductDetailsScreen, reading the id from state.pathParameters.
- When tapping a product, navigate with context.push('/product/\${product.id}', extra: product).
- In ProductDetailsScreen's builder, read the full product object from state.extra and display its full details (not just the id).

PRACTICE HINT

The path parameter tells you which product, while extra avoids needing to look it up again from a list — both are commonly used together.

ShellRoute & Persistent Bottom Navigation

1. What is ShellRoute?

ShellRoute wraps a set of child routes with a shared, persistent UI shell (like a Scaffold with a BottomNavigationBar), so that shell stays on screen while only the inner content changes as the user navigates between child routes.

2. Persistent Bottom Navigation

Because the ShellRoute's builder wraps every child route, a BottomNavigationBar placed in that shell remains visible and in the same state across all child screens, rather than being rebuilt or hidden during navigation.

SHELL ROUTE WRAPPER (Persistent UI Screen)

Inner Viewport: child Widget
(Swaps HomeScreen, FavoritesScreen, ProfileScreen
without rebuilding the outer ShellRoute Scaffold)

Persistent Navigation Bar (Doesn't animate or reload on tab switch)

Nested stacks & ShellRoute use-cases

3. Nested navigation

ShellRoute supports its own nested routes, meaning each tab in the shell can have its own internal navigation stack (e.g. pushing a details screen within the 'Movies' tab) while the outer shell (nav bar) stays fixed.

4. When to use ShellRoute

Use ShellRoute whenever multiple routes should share a persistent piece of UI — most commonly a bottom navigation bar or a side navigation drawer — that shouldn't be rebuilt or removed as the user moves between those routes.

ShellRoute Nested Router

```
// ShellRoute configuration
ShellRoute(
  builder: (context, state, child) {
    return ScaffoldWithNavbar(child: child);
  },
  routes: [
    GoRoute(
      path: '/movies',
      builder: (context, state) => MoviesScreen(),
      routes: [
        GoRoute(
          path: 'details/:id',
          builder: (context, state) => DetailsScreen(),
        ),
      ],
    ),
    GoRoute(path: '/favs', ...),
  ]
)
```

ShellRoute Conversion

Convert a Bottom Navigation app into a ShellRoute implementation while keeping the navigation bar visible across screens.

Requirements

- Wrap the Home, Favorites, and Profile GoRoutes inside a single ShellRoute.
- Implement the shell's builder to return a Scaffold containing a BottomNavigationBar and the given child widget as its body.
- Wire the BottomNavigationBar's onTap to navigate using context.go() to each tab's path.
- Confirm the navigation bar remains visible and correctly highlights the active tab as you move between Home, Favorites, and Profile.

PRACTICE HINT

The BottomNavigationBar's currentIndex should be derived from the current route path (e.g. by checking state.uri or GoRouterState.of(context).matchedLocation) rather than a manually tracked variable.

Movie Explorer

Build a complete application using GoRouter that combines every navigation concept covered across both parts of the course.

What You'll Build (Core Architecture):

- Splash Screen
- Login Screen
- Home Screen
- Bottom Navigation with: Movies, Favorites, Profile
- Movie Details screen using a route parameter
- Settings screen using a named route
- Pass the selected movie using extra
- Use `context.go()` and `context.push()` appropriately
- Implement the Bottom Navigation using `ShellRoute` so it remains visible while navigating between tabs

CONGRATULATIONS!

Flutter Navigation Mastered

You've mastered the built-in Navigator stack, complex tab bar layouts, and the modern declarative GoRouter with nested ShellRoute. Next up: MVVM Architecture & State Management BLOC!

- Push/Pop Stacks

- Tab Controllers

- IndexedStack TabState

- GoRouter Config

- Path/Query Parameters

- Nested ShellRoutes